

Technical Architecture

The document viewer as a product a third-party common data environment embeds.

Status. The rendering core is built and boundary-enforced. The integration surface is not. Section 12 states exactly what is missing — read it before quoting anything here to a customer.

Scope. The viewer as described by ADR 12 (cde-platform, docs/adr/0012-viewer-as-a-standalone-product.md).

Companions. viewer-integration-guide.md for how a host uses it; viewer-extraction-inventory.md for what still has to move; and the platform's own docs/architecture.md, which remains the authority on tenancy, the ISO 19650 state machine and the request lifecycle.

Contents

1. What this is, and what it is not

It is a browser-side viewer for construction documents — PDF, DXF/DWG drawings, and IFC models — with a markup, measurement, redaction and signature layer over the top, plus the server-side conversion needed to turn formats a browser cannot read into ones it can.

It is not a document management system. It does not own documents, users, projects, permissions or an audit trail. The host CDE owns all of those, and the viewer's job is to borrow a document briefly, render it, and hand back what the user drew on it.

That distinction is the whole architecture. Every design decision below follows from “the bytes belong to someone else”.

2. Three layers, and the one that matters

HOST CDE	EXCHANGE	VIEWER PRODUCT
Their document store SharePoint / S3 / Blob	1. mount the viewer ✗ NOT BUILT — no embed surface	viewer-core 6 components · 6 services
Their identity provider Entra / Okta / own	2. identify the user ✗ NOT BUILT — no token exchange	Conversion pipeline fetch → scan → render
Their web interface where the viewer appears	3. hand over the document → ✓ BUILT POST /api/conversions	Markup store tied to our user table
	← 4. return the markups ✗ NOT BUILT — 17 operations undecided	

Figure 1 — Three of the four exchanges do not exist. The viewer can already take a document from anyone's storage; it cannot yet be mounted, be told who is looking, or give anything back.

src/viewer-core/ is the product. 3,196 lines of production TypeScript across 12 source files, with 7 spec files beside them. Nothing in it imports anything from the application — not a service, not a model, not an environment constant. When the separate repository exists, this directory is **copied, not untangled**.

The application shell above it is this platform's own consumer of the viewer. It is not part of the product; it is the first integrator, and useful precisely because an awkward contract will be awkward for us first.

2.1 The boundary is asserted, not intended

`viewer-core.boundary.spec.ts` reads every `./*.ts` in the directory through Vite's `import.meta.glob` and fails on four conditions.

Assertion	Why it exists
At least 6 source files present	A renamed directory would otherwise make every check below vacuously true
Nothing imports <code>app/</code> , <code>core/</code> , <code>features/</code> , <code>shared/</code> , <code>environments/</code>	The dependency that breaks extraction

Assertion	Why it exists
Nothing reaches above the directory except ../testing/ from a spec	Catches the same thing if a folder is renamed
No production file reaches outside at all	Stops the test-helper exemption becoming a hole

Table 1 — The four boundary assertions.

One `inject(AuthService)` added in a hurry turns a copy into a migration, and **nothing else in the build would notice** — the application compiles perfectly well with the dependency pointing the wrong way. That is why this is a test and not a convention.

Components sit flat beside the services rather than in a `components/` subdirectory. The glob is `./*.ts` and the rule forbids every `../` import, so a nested component reaching back for `../viewer-state.service` would both escape the glob and trip the rule it escaped.

3. viewer-core

3.1 Services

Service	Lines	Responsibility
viewer-state.service	375	The single source of truth. Signals for document, page, zoom, rotation, active tool, selection, sidebar tab, and the version-commit token that makes panels reload after a server-side operation
markup-engine.service	565	All drawing: pointer events (mouse and touch), shape creation, SVG rendering, hit testing
pdf-engine.service	201	pdfjs-dist wrapper — document loading, page rendering to canvas, text-layer extraction
measurement.service	165	Scale calibration and the geometry behind length, area and radius. <code>unitsPerPixel === 1</code> with unit <code>px</code> means uncalibrated
outline.service	147	A PDF's bookmarks and its link annotations
drawing-search.service	129	Text search across a converted drawing's labels

Table 2 — The six server-independent services.

3.2 Components

`cad-viewer` (the DXF/DWG canvas), `ifc-tree` (the model hierarchy), `outline-panel`, `page-links`, `tool-rail`, and `icon` — the last being the product's whole icon set as stroke-only 24×24 path data, so one definition sits on a light rail, a dark header and an accent-filled button without variants.

`cad-viewer.component.ts` is 629 lines, over the §3.3 limits of 400 per file and 200 per component. Recorded rather than hidden: it is known debt, it predates the extraction, and splitting a canvas component wants a test suite that can actually run first.

3.3 What is deliberately not here

The 15 components still in the application are not there by accident. Thirteen import the platform's HTTP services; two (`viewer.component`, `viewer3d.component`) inject `ViewerService`,

which calls `/api/viewer/{id}` and the annotation endpoints. Where those operations end up — viewer-side work against the fetched copy, a callback into the host, or out of the product — is ADR 12's open step 4. Moving them before that decision would answer it by accident.

4. Three rendering pipelines

PDF is rendered in the browser by `pdfjs-dist` 6. Text layer extracted for search and redaction; markup drawn as SVG in a layer above the canvas so it scales with zoom without re-rasterising.

DXF/DWG cannot be rendered by the browser at all. The converter turns a DXF into SVG server-side; DWG is converted to DXF first. The result is an SVG the browser renders directly, with an invisible text layer over it for search — the same trick the PDF viewer uses, for the same reason.

IFC is rendered with `three 0.185` as WebGL geometry, with `ifc-tree` carrying the model hierarchy alongside it. **The tree is the primary interface and the canvas is the visual layer over it**, not the other way round — a WebGL canvas cannot be made WCAG-conformant on its own, and §1A.4 requires an equivalent accessible route to the same information. Building the tree first is what makes that true rather than aspirational.

4.1 The same drawing, rendered twice, on purpose

A DXF is converted twice, by two different paths, and the reason is not obvious enough to leave undocumented.

For the viewer, geometry is drawn as SVG paths and the text is emitted as `<text>` with `fill="none"` — invisible, positioned, selectable. The browser draws the glyphs from the geometry; the invisible layer exists only so search and selection have something to hit.

For PDF export, that trick is inverted. LibreOffice discards SVG text it cannot see, so an invisible layer would produce a PDF with no extractable text at all. The print render therefore draws **no** glyph geometry (`TextPolicy.IGNORE`) and emits **visible** `<text>`, which LibreOffice turns into real, searchable PDF text.

The consequence that bites: calibration must measure what was actually drawn. The viewer render includes glyph paths, so it measures the whole layout; the print render draws no text, so it measures geometry only. Using the wrong set silently shifts every label on the sheet.

A related trap, recorded because it cost a day: rendering white-on-white passes every test that checks the PDF exists, has pages, and contains text. Only rasterising the output and measuring ink coverage catches it, and that check now lives in the converter's test suite.

5. Why conversion is server-side and stays there

Office documents go through LibreOffice, DWG through LibreDWG or the ODA File Converter, OCR through Tesseract, DXF through `ezdxf`. None of these run in a browser, and all of them are heavy native parsers on untrusted input — §5.13.10 requires them in a sandboxed, resource-limited, network-isolated worker with a timeout, out of process.

This is the constraint that rules out the otherwise-attractive design where the browser fetches straight from the customer's storage and we never see the file: **that architecture cannot render a .docx at all.**

5.1 DWG has no clean path in a distributed product

	LibreDWG dwg2dxf	ODA File Converter
Licence	GPL-3.0	Proprietary
In the image	Yes	No
The problem	We ship it and owe every recipient corresponding source, which we do not provide	We cannot ship it, so a customer install has no DWG unless they obtain one themselves

Table 3 — Two converters, opposite licence problems.

Running our own SaaS conceals this — the image never leaves us. The moment the product is installed by a customer, both problems are real on every install. Recorded as ADR 13 (cde-platform, docs/adr/0013-dwg-conversion-in-a-distributed-product.md), referred to counsel, unresolved.

A supplied ODA is fully supported: discovered from a directory or a binary, launched inside a virtual framebuffer (it is a Qt application and opens a display even converting from the command line), probed once at startup, and reported as `odaRunnable` distinctly from `odaInstalled`.

6. Document ingress

The one integration exchange that is built. The host mints a short-lived URL with its own credentials and posts it; the viewer fetches once, converts, and discards.

#	STAGE	WHAT HAPPENS
1	Host mints a link	Short-lived, single-object, your credentials. We never hold them.
2	Submit	POST /api/conversions { sourceUrl, targetFormat } — 202 + Location header, under one second
3	Destination policy	Resolves the hostname and validates the ADDRESS, not the string. Refuses loopback, link-local, 169.254.169.254, RFC 1918, .internal. Redirects are NOT followed.
4	Bounded fetch	Size cap enforced during the stream, not after. Connect and read timeouts.
5	Quarantine + scan	Magic-byte type check, then ClamAV. Not referenced by the application until clean.
6	Convert	Sandboxed, resource-limited, network-isolated worker, out of process.
7	Poll	GET /api/conversions/{jobId} — PENDING, RUNNING, then SUCCEEDED / FAILED / CANCELLED (terminal)
8	Collect	GET /api/conversions/{jobId}/content — streamed

Figure 2 — The one exchange that is built, end to end. Stages 1, 7 and 8 are yours; 2 to 6 are ours. Always send an Idempotency-Key so a timed-out retry returns the same job rather than converting twice.

A refused destination is a **422**, and a job belonging to another tenant is a **404** rather than a 403 — invisible, not forbidden. Every operation requires `document:convert`.

The viewer never holds a customer credential and never learns which storage platform the URL points at — SharePoint, S3, Azure Blob and GCS all reduce to the same code path. That property is what makes “integrates with any CDE” a design rather than a slogan.

The source URL is never stored. It is a bearer credential with a short life; persisting it would turn the job table into a credential store. Only its host is kept on the job record.

The address is checked twice, deliberately: cheaply on submission so an obviously wrong value fails fast with a clear message, and again against the resolved address at fetch time, which is the check that actually decides. No pattern can validate what a name will resolve to.

6.1 The gap under this

The viewer’s own front end has never called `/api/conversions`. It still reads `/api/viewer/{id}` and `/api/documents/*`, which assume the platform owns the document. The product has two halves that do not meet: an ingress for the host’s storage with no interface on it, and an interface that only opens documents we already hold.

7. State

Angular signals throughout, with `ViewerStateService` as the single store. No `NgRx`, no observable soup: the viewer’s state is small, synchronous and local — zoom, page, tool, selection — and a store framework would add indirection without adding capability.

Server-side operations that rewrite the document (redaction, OCR, flatten, form fill, page rearrange) commit a **new version** and bump a reload token. Panels watch the token rather than each other, which is what lets those operations compose: each starts from the previous one’s output rather than from the untouched original.

8. Accessibility architecture

Not a layer applied afterwards. Three decisions are structural.

- **The IFC tree is the primary data interface.** A canvas alone cannot conform.
- **Every drag interaction needs a single-pointer alternative** (SC 2.5.7) — which for a markup tool means every shape must be creatable and movable without a drag. This is a known gap: the callout box cannot currently be dragged at all, because `updateShape` has no callout case.
- **Exports must be tagged and PDF/UA-conformant.** An inaccessible export is a product accessibility failure, and it is the most common gap found in government audits.

9. Security posture

Most of §5 is inherited from the platform and documented there — RLS tenant isolation, the hash-chained audit trail, PBKDF2 password storage, the §5.4 response headers. What the **viewer product** must own itself once distributed is the question this table answers.

Control	Where it lives now	Where it must live
SSRF policy on the ingress	Platform fetch/ package	Travels with the product — it is the product’s own attack surface
Upload magic-byte + AV	Platform	Travels
Content Security Policy	Platform response headers	Must become per-deployment, because embedding requires relaxing frame-ancestors
Tenant isolation	Platform RLS	Does NOT travel. The host owns tenancy; the viewer must not assume it
Audit	Platform hash chain	Host’s concern; the viewer emits events, it does not store them

Table 4 — What travels with the product and what does not.

The last two rows matter most. A distributed viewer that assumes it owns tenancy would be wrong in a way that is expensive to unwind.

10. Performance

§7.1 applies unchanged: every interactive request under a second, bulk work async with a job id returned in under a second. Conversion is bulk by definition.

Frontend budgets — LCP < 2.0 s, INP < 200 ms, CLS < 0.1, initial bundle < 250 KB gzipped — are the ones the viewer can actually breach on its own. The PDF and 3D renderers are the risk: both are lazy-loaded via `dynamic import()` so they never enter the initial bundle.

Large files never touch application memory. A 2 GB IFC streams to object storage and is processed out of process; §7.7’s prohibition on `readAllBytes` over user content is absolute.

11. Build and verification

Angular 22 with TypeScript strict **and** `noUncheckedIndexedAccess`. Third-party JavaScript is bundled, never loaded from a CDN — §5.12 A08, and it is also what makes air-gapped deployment possible.

A note on the current environment, because it affects what any statement about test results is worth: `ng test` and `ng build` require Node $\geq 22.22.3$, and the development container runs 22.22.2. Specs that need Angular TestBed cannot run there at all; `tsc --noEmit` and Vitest-only specs can. Any claim of “tests pass” in this repository should say which of those two it means.

12. What this architecture does not yet have

Stated plainly, because the gap between “the viewer works” and “a CDE can integrate it” is larger than a demo suggests.

No embed surface. No `postMessage`, no custom elements, no `@angular/elements`. The CSP sets `frame-ancestors 'none'` and the CORS source registers no origin unless a deployment names one, so cross-origin framing and cross-origin XHR are both closed. The only honest answer today is “serve our build from your own origin”, which is a deployment, not an integration.

No identity contract. Authentication is our own JWT from /api/auth/login against our own user table — no OIDC, no SAML, no API key, no token exchange.

TODAY	NEEDED
Priya, at Asite host identity	Priya, at Asite signed assertion
↓ blocked	↓
Annotation.author @ManyToOne User	Annotation.authorRef opaque value
↓	↓
OUR users table she must exist as a row	Issuer + subject no row required
Integration means shadow-provisioning every host user into our database.	The host stays the system of record for who its people are.

Figure 3 — The difference is one field. Annotation.author is a foreign key into our own user table, so a markup cannot exist without a local user row.

No external document identity. Document has no externalId, so a host must store our numeric id against its own record.

36 endpoints across 13 services, re-derived by following the imports from every viewer component. (viewer-extraction-inventory.md §2 says 26 across 9; that number is stale and the file is the older count, not a different scope.) Five of those are content, and are what the ingress replaces. Seventeen are document operations awaiting ADR 12 step 4. The rest are identity and collaboration.

Not a publishable package. viewer-core has no package.json or ng-package.json — it is a directory, not a library.

No attribution file. No LICENSE, NOTICE or THIRD-PARTY-NOTICES.txt at this repository's root. §17.2 makes shipping that file a licence obligation, so first distribution without it is a breach rather than an untidiness.

No DWG position. ADR 13, above.

Of these, the embed contract and the identity contract are **decisions nobody has taken**, not work nobody has done — and everything else queues behind them.